

实验三报告

姓名：李祥熙 学号：2234412799

2026 年 5 月 20 日

1 实验目的

本实验的目标如下：

- 目标一：掌握 SQLite、MySQL、PostgreSQL 的连接与建表流程
- 目标二：使用 Rust 实现插入、查询与库存扣减等核心逻辑
- 目标三：使用视图实现统计报表，并通过日期范围进行查询

2 实验过程

2.1 实验环境

- 操作系统：macOS
- 数据库：SQLite 3、MySQL 8.4 (Docker)、PostgreSQL 16 (Docker)
- 其他关键工具：Docker + Colima、cargo

2.2 实现步骤

1. **环境准备与数据库启动**：使用 Docker 启动 MySQL/PostgreSQL 容器，配置默认连接地址。
2. **任务一 (SQLite)**：创建 users 表，实现注册、查询与列表功能。
3. **任务二 (MySQL)**：创建 app_logs 表，写入不同等级日志并查询 24 小时内 ERROR。
4. **任务三 (PostgreSQL)**：创建 products 表，实现库存扣减与低库存查询。
5. **任务四 (MySQL)**：创建 orders 表与视图 daily_sales_report，完成日期范围统计。

3 实验内容

3.1 题目

- 任务一：用户注册系统 (SQLite)
 - 创建一个 SQLite 数据库 users.db
 - 建立 users 表 (id 主键、username 唯一、password、created_at 时间戳)
 - 编写 Rust 程序实现用户注册功能，插入测试数据 (至少 5 条)
 - 实现按用户名查询用户信息的功能
- 任务二：日志记录系统 (MySQL)
 - 连接本地 MySQL 数据库创建 app_logs 表
 - 字段包含：log_id (自增)、log_level (ENUM)、content、created_at
 - 编写日志写入函数，支持插入不同级别的日志
 - 实现查询最近 24 小时内的 ERROR 级别日志
- 任务三：商品库存管理 (PostgreSQL)
 - 在 PostgreSQL 中创建 products 表
 - 字段：商品 ID、名称、库存数量、最后更新时间
 - 实现库存扣减功能 (购买商品后更新库存)
 - 编写查询库存量低于 10 件的商品功能
- 任务四：数据统计报表 (MySQL)
 - 基于订单表生成每日销售报表
 - 统计：订单总数、销售总额、平均客单价
 - 使用数据库视图存储统计逻辑
 - 实现按日期范围查询统计结果

3.2 任务一：用户注册系统 (SQLite)

实现要点：使用 rusqlite 连接本地 users.db，初始化 users 表，并提供 seed、register、query、list 命令。

主要参数：

- 用户名：username，唯一约束
- 密码：password，明文示例

- 创建时间: `created_at`, 默认 `CURRENT_TIMESTAMP`

关键代码片段 (建表 + 插入):

```
CREATE TABLE IF NOT EXISTS users (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    username TEXT NOT NULL UNIQUE,  
    password TEXT NOT NULL,  
    created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP  
);  
  
INSERT OR IGNORE INTO users (username, password, created_at)  
VALUES (?1, ?2, CURRENT_TIMESTAMP);
```

说明: 建表语句保证用户名唯一; 插入语句使用 `OR IGNORE` 避免重复插入报错。

关键代码片段 (查询与列表):

```
fn query_user(conn: &Connection, username: &str) -> rusqlite::Result<  
    Option<User>> {  
    let mut stmt = conn.prepare(  
        "SELECT id, username, password, created_at FROM users  
        WHERE username = ?1",  
    )?;  
    let mut rows = stmt.query(params![username])?;  
    if let Some(row) = rows.next()? {  
        Ok(Some(User {  
            id: row.get(0)?,  
            username: row.get(1)?,  
            password: row.get(2)?,  
            created_at: row.get(3)?,  
        }))  
    } else {  
        Ok(None)  
    }  
}
```

说明: 通过参数化查询避免 SQL 注入; 当不存在匹配记录时返回 `None`, 便于命令行输出友好提示。

3.3 任务二: 日志记录系统 (MySQL)

实现要点: 使用 `mysql crate` 建表, 并提供日志写入函数与 “24 小时内 `ERROR` 日志” 查询。

主要参数:

- 日志等级：INFO、WARN、ERROR (枚举)
- 内容：content，文本
- 时间：created_at，默认 CURRENT_TIMESTAMP

关键代码片段 (建表与插入):

```
CREATE TABLE IF NOT EXISTS app_logs (  
    log_id INT AUTO_INCREMENT PRIMARY KEY,  
    log_level ENUM('INFO','WARN','ERROR') NOT NULL,  
    content TEXT NOT NULL,  
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP  
);  
  
INSERT INTO app_logs (log_level, content) VALUES (:level, :content);
```

说明：log_level 使用 ENUM 限定范围；插入使用命名参数，避免拼接字符串。

关键代码片段 (24 小时内 ERROR 查询):

```
SELECT log_id, log_level, content,  
       DATE_FORMAT(created_at, '%Y-%m-%d %H:%i:%s') AS created_at  
FROM app_logs  
WHERE log_level = 'ERROR'  
      AND created_at >= NOW() - INTERVAL 24 HOUR  
ORDER BY created_at DESC;
```

说明：时间过滤直接在 SQL 层完成，确保输出只包含最近 24 小时的 ERROR 记录。

3.4 任务三：商品库存管理 (PostgreSQL)

实现要点：创建 products 表，并通过 UPDATE ... RETURNING 原子化扣减库存。

主要参数：

- 商品编号：product_id, SERIAL 自增
- 库存：stock, 整数
- 更新时间：updated_at, NOW()

关键代码片段 (扣减库存):

```
UPDATE products  
SET stock = stock - $1,  
    updated_at = NOW()  
WHERE product_id = $2
```

```
    AND stock >= $1
RETURNING product_id, name, stock, to_char(updated_at, 'YYYY-MM-DD HH24
:MI:SS');
```

说明：在单条 SQL 中完成库存校验与扣减，避免并发场景下出现负库存。

关键代码片段（低库存查询）：

```
SELECT product_id, name, stock, to_char(updated_at, 'YYYY-MM-DD HH24:MI
:SS')
FROM products
WHERE stock < $1
ORDER BY stock ASC;
```

说明：阈值由命令行输入，默认 10，按库存从低到高排序输出。

3.5 任务四：数据统计报表（MySQL）

实现要点：通过视图 daily_sales_report 存储统计逻辑，按日期范围查询。

主要参数：

- 订单金额：amount，DECIMAL(10,2)
- 统计字段：订单数、总金额、平均金额

关键代码片段（创建视图）：

```
CREATE OR REPLACE VIEW daily_sales_report AS
SELECT DATE(created_at) AS report_date,
       COUNT(*) AS order_count,
       SUM(amount) AS total_amount,
       AVG(amount) AS avg_amount
FROM orders
GROUP BY DATE(created_at);
```

说明：视图固化统计逻辑，查询时只需筛选日期范围即可。

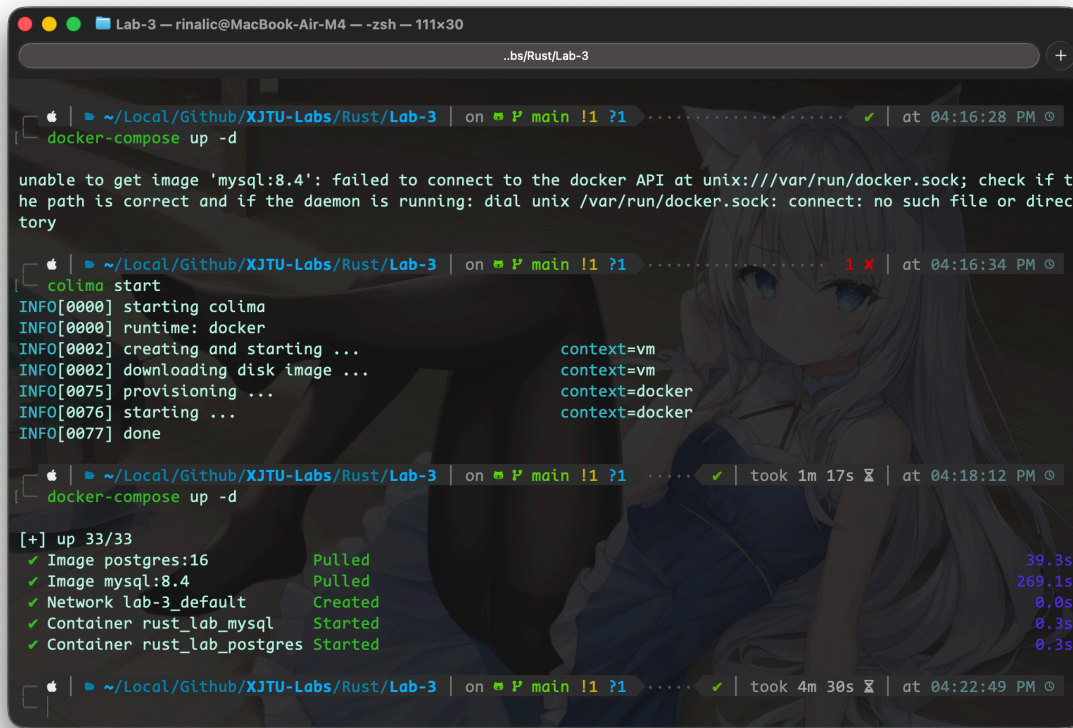
关键代码片段（范围查询）：

```
SELECT DATE_FORMAT(report_date, '%Y-%m-%d') AS report_date,
       order_count, total_amount, avg_amount
FROM daily_sales_report
WHERE report_date BETWEEN :start_date AND :end_date
ORDER BY report_date;
```

说明：日期范围由命令行传入，用于生成指定时间段的报表。

4 实验结果

4.1 环境启动结果



```
Lab-3 -- rinalic@MacBook-Air-M4 -- zsh -- 111x30
..bs/Rust/Lab-3

[+] up 33/33
✓ Image postgres:16 Pulled 39.3s
✓ Image mysql:8.4 Pulled 269.1s
✓ Network lab-3_default Created 0.0s
✓ Container rust_lab_mysql Started 0.3s
✓ Container rust_lab_postgres Started 0.3s

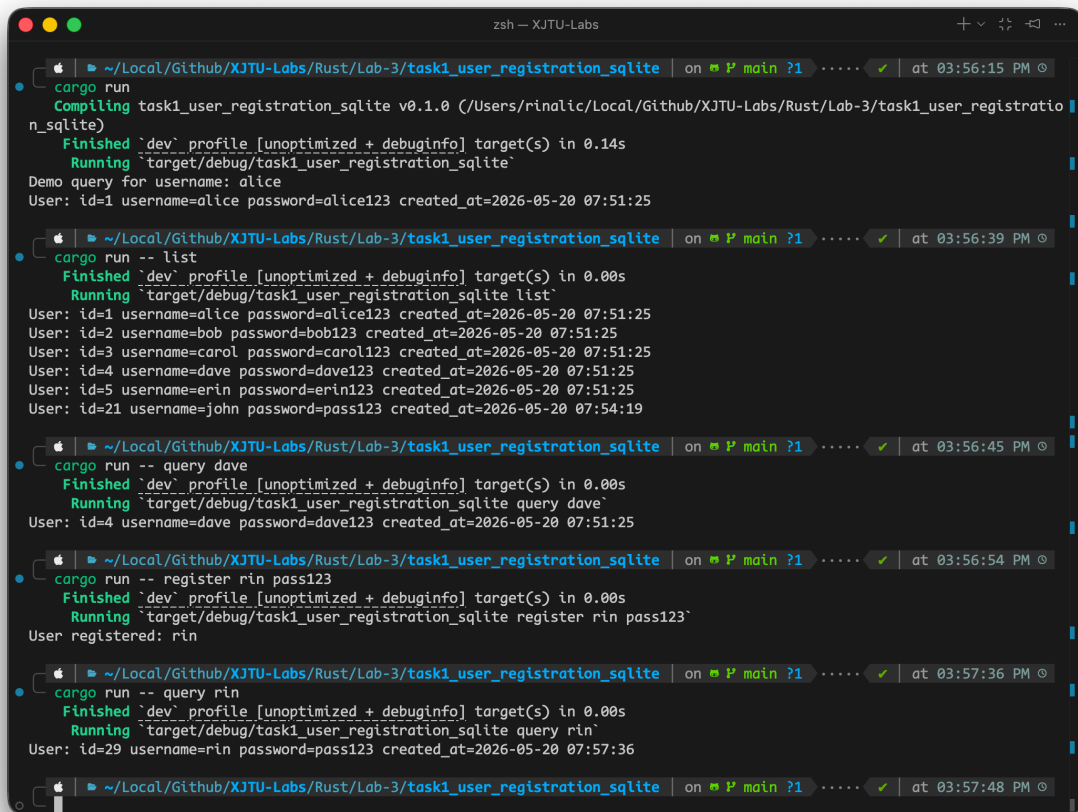
[+] up 33/33
✓ Image postgres:16 Pulled 39.3s
✓ Image mysql:8.4 Pulled 269.1s
✓ Network lab-3_default Created 0.0s
✓ Container rust_lab_mysql Started 0.3s
✓ Container rust_lab_postgres Started 0.3s

[+] up 33/33
✓ Image postgres:16 Pulled 39.3s
✓ Image mysql:8.4 Pulled 269.1s
✓ Network lab-3_default Created 0.0s
✓ Container rust_lab_mysql Started 0.3s
✓ Container rust_lab_postgres Started 0.3s
```

图 1: Docker 启动 MySQL 与 PostgreSQL

说明：图中显示两种数据库容器成功启动，端口 3306 和 5432 已映射到本机，保证后续任务可以直接通过默认地址连接。

4.2 任务一运行结果



```
zsh - XJTU-Labs

~/Local/Github/XJTU-Labs/Rust/Lab-3/task1_user_registration_sqlite on P main ?1 ..... at 03:56:15 PM
cargo run
Compiling task1_user_registration_sqlite v0.1.0 (/Users/rinalic/Local/Github/XJTU-Labs/Rust/Lab-3/task1_user_registration_sqlite)
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.14s
Running `target/debug/task1_user_registration_sqlite`
Demo query for username: alice
User: id=1 username=alice password=alice123 created_at=2026-05-20 07:51:25

~/Local/Github/XJTU-Labs/Rust/Lab-3/task1_user_registration_sqlite on P main ?1 ..... at 03:56:39 PM
cargo run -- list
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.00s
Running `target/debug/task1_user_registration_sqlite list`
User: id=1 username=alice password=alice123 created_at=2026-05-20 07:51:25
User: id=2 username=bob password=bob123 created_at=2026-05-20 07:51:25
User: id=3 username=carol password=carol123 created_at=2026-05-20 07:51:25
User: id=4 username=dave password=dave123 created_at=2026-05-20 07:51:25
User: id=5 username=erin password=erin123 created_at=2026-05-20 07:51:25
User: id=21 username=john password=pass123 created_at=2026-05-20 07:54:19

~/Local/Github/XJTU-Labs/Rust/Lab-3/task1_user_registration_sqlite on P main ?1 ..... at 03:56:45 PM
cargo run -- query dave
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.00s
Running `target/debug/task1_user_registration_sqlite query dave`
User: id=4 username=dave password=dave123 created_at=2026-05-20 07:51:25

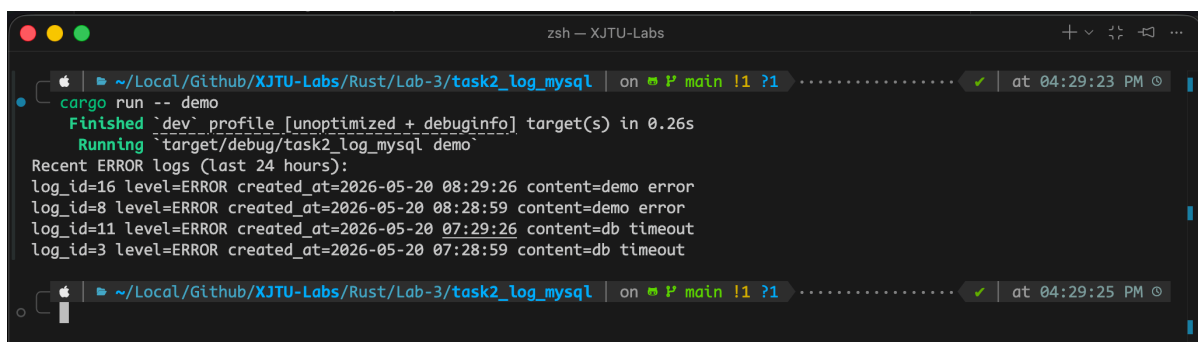
~/Local/Github/XJTU-Labs/Rust/Lab-3/task1_user_registration_sqlite on P main ?1 ..... at 03:56:54 PM
cargo run -- register rin pass123
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.00s
Running `target/debug/task1_user_registration_sqlite register rin pass123`
User registered: rin

~/Local/Github/XJTU-Labs/Rust/Lab-3/task1_user_registration_sqlite on P main ?1 ..... at 03:57:36 PM
cargo run -- query rin
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.00s
Running `target/debug/task1_user_registration_sqlite query rin`
User: id=29 username=rin password=pass123 created_at=2026-05-20 07:57:36
```

图 2: SQLite 用户注册与查询演示

说明：演示命令完成了用户数据的种子插入与按用户名查询，输出包含 id、username、password 和 created_at 字段，符合题目要求。

4.3 任务二运行结果



```
zsh - XJTU-Labs

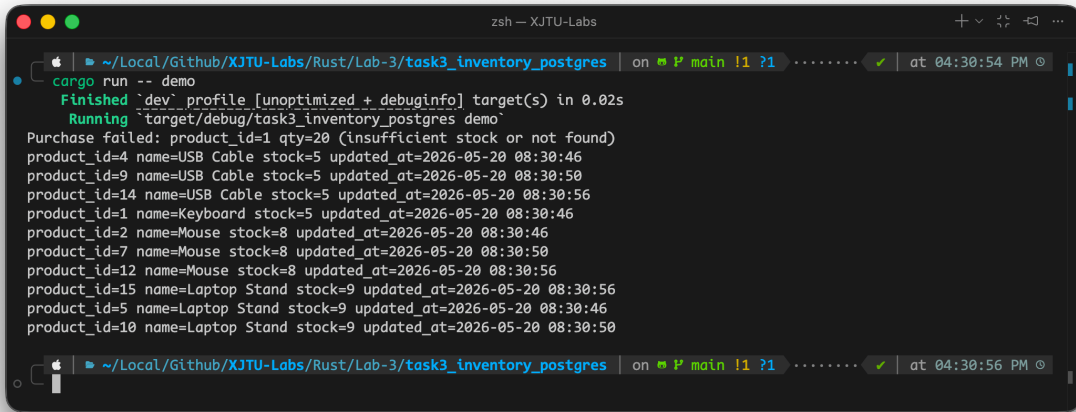
~/Local/Github/XJTU-Labs/Rust/Lab-3/task2_log_mysql on P main !1 ?1 ..... at 04:29:23 PM
cargo run -- demo
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.26s
Running `target/debug/task2_log_mysql demo`
Recent ERROR logs (last 24 hours):
log_id=16 level=ERROR created_at=2026-05-20 08:29:26 content=demo error
log_id=8 level=ERROR created_at=2026-05-20 08:28:59 content=demo error
log_id=11 level=ERROR created_at=2026-05-20 07:29:26 content=db timeout
log_id=3 level=ERROR created_at=2026-05-20 07:28:59 content=db timeout

~/Local/Github/XJTU-Labs/Rust/Lab-3/task2_log_mysql on P main !1 ?1 ..... at 04:29:25 PM
```

图 3: MySQL 日志写入与 ERROR 查询

说明：演示中写入多级别日志，并成功过滤出最近 24 小时内的 ERROR 记录，输出包含 log_id、log_level、content 与 created_at。

4.4 任务三运行结果



```

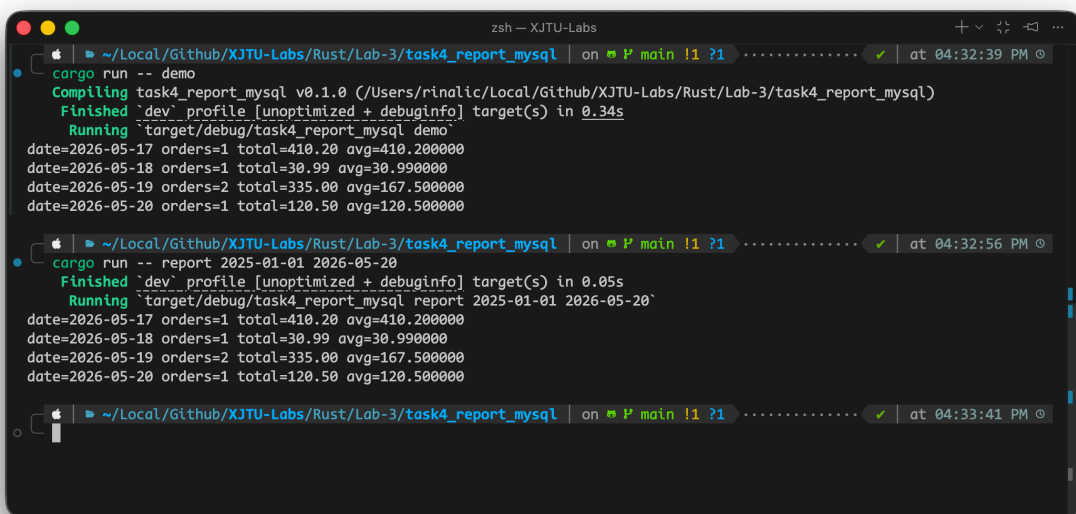
zsh - XJTU-Labs
~/Local/Github/XJTU-Labs/Rust/Lab-3/task3_inventory_postgres on P main !1 ?1 ..... at 04:30:54 PM
cargo run -- demo
Finished dev profile [unoptimized + debuginfo] target(s) in 0.02s
Running target/debug/task3_inventory_postgres demo
Purchase failed: product_id=1 qty=20 (insufficient stock or not found)
product_id=4 name=USB Cable stock=5 updated_at=2026-05-20 08:30:46
product_id=9 name=USB Cable stock=5 updated_at=2026-05-20 08:30:50
product_id=14 name=USB Cable stock=5 updated_at=2026-05-20 08:30:56
product_id=1 name=Keyboard stock=5 updated_at=2026-05-20 08:30:46
product_id=2 name=Mouse stock=8 updated_at=2026-05-20 08:30:46
product_id=7 name=Mouse stock=8 updated_at=2026-05-20 08:30:50
product_id=12 name=Mouse stock=8 updated_at=2026-05-20 08:30:56
product_id=15 name=Laptop Stand stock=9 updated_at=2026-05-20 08:30:56
product_id=5 name=Laptop Stand stock=9 updated_at=2026-05-20 08:30:46
product_id=10 name=Laptop Stand stock=9 updated_at=2026-05-20 08:30:50

```

图 4: PostgreSQL 库存扣减与低库存查询

说明：演示中对指定商品进行库存扣减，并输出低于阈值的商品列表，验证了库存更新和低库存查询逻辑。

4.5 任务四运行结果



```

zsh - XJTU-Labs
~/Local/Github/XJTU-Labs/Rust/Lab-3/task4_report_mysql on P main !1 ?1 ..... at 04:32:39 PM
cargo run -- demo
Compiling task4_report_mysql v0.1.0 (/Users/rinalic/Local/Github/XJTU-Labs/Rust/Lab-3/task4_report_mysql)
Finished dev profile [unoptimized + debuginfo] target(s) in 0.34s
Running target/debug/task4_report_mysql demo
date=2026-05-17 orders=1 total=410.20 avg=410.200000
date=2026-05-18 orders=1 total=30.99 avg=30.990000
date=2026-05-19 orders=2 total=335.00 avg=167.500000
date=2026-05-20 orders=1 total=120.50 avg=120.500000

~/Local/Github/XJTU-Labs/Rust/Lab-3/task4_report_mysql on P main !1 ?1 ..... at 04:32:56 PM
cargo run -- report 2025-01-01 2026-05-20
Finished dev profile [unoptimized + debuginfo] target(s) in 0.05s
Running target/debug/task4_report_mysql report 2025-01-01 2026-05-20
date=2026-05-17 orders=1 total=410.20 avg=410.200000
date=2026-05-18 orders=1 total=30.99 avg=30.990000
date=2026-05-19 orders=2 total=335.00 avg=167.500000
date=2026-05-20 orders=1 total=120.50 avg=120.500000

~/Local/Github/XJTU-Labs/Rust/Lab-3/task4_report_mysql on P main !1 ?1 ..... at 04:33:41 PM

```

图 5: MySQL 日销售报表视图与范围查询

说明：演示命令先插入订单数据，再通过视图查询给定日期范围内的每日统计，输出包含订单数、总额与平均客单价。

5 总结

本实验成功完成了四个数据库任务，具体包括：

- 使用 SQLite 完成用户注册、查询和列表功能
- 使用 MySQL 完成日志写入与 24 小时 ERROR 查询
- 使用 PostgreSQL 完成库存扣减与低库存筛选
- 使用 MySQL 视图完成日销售统计与范围查询

经验与建议：使用参数化 SQL 与视图可以提升安全性与可维护性；在演示中加入 seed 数据可以显著提高复现实验结果的稳定性。

附录：实验源代码

A.1 任务一源代码

```
use rusqlite::{params, Connection};
use std::env;
use std::error::Error;

#[derive(Debug)]
struct User {
    id: i64,
    username: String,
    password: String,
    created_at: String,
}

fn init_db(conn: &Connection) -> rusqlite::Result<()> {
    conn.execute(
        "CREATE TABLE IF NOT EXISTS users (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            username TEXT NOT NULL UNIQUE,
            password TEXT NOT NULL,
            created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
        )",
    ),
}
```

```

        [],
    )?;
    Ok(())
}

fn seed_users(conn: &Connection) -> rusqlite::Result<usize> {
    let samples = [
        ("alice", "alice123"),
        ("bob", "bob123"),
        ("carol", "carol123"),
        ("dave", "dave123"),
        ("erin", "erin123"),
    ];

    let mut inserted = 0;
    for (username, password) in samples {
        let changes = conn.execute(
            "INSERT OR IGNORE INTO users (username, password,
                created_at)
            VALUES (?1, ?2, CURRENT_TIMESTAMP)",
            params![username, password],
        )?;
        inserted += changes;
    }

    Ok(inserted)
}

fn register_user(conn: &Connection, username: &str, password: &str) ->
rusqlite::Result<bool> {
    let changes = conn.execute(
        "INSERT OR IGNORE INTO users (username, password, created_at)
        VALUES (?1, ?2, CURRENT_TIMESTAMP)",
        params![username, password],
    )?;
    Ok(changes > 0)
}

fn query_user(conn: &Connection, username: &str) -> rusqlite::Result<
Option<User>> {
    let mut stmt = conn.prepare(

```

```

        "SELECT id, username, password, created_at
        FROM users
        WHERE username = ?1",
    )?;
    let mut rows = stmt.query(params![username])?;
    if let Some(row) = rows.next()? {
        Ok(Some(User {
            id: row.get(0)?,
            username: row.get(1)?,
            password: row.get(2)?,
            created_at: row.get(3)?,
        }))
    } else {
        Ok(None)
    }
}

fn list_users(conn: &Connection) -> rusqlite::Result<Vec<User>> {
    let mut stmt = conn.prepare(
        "SELECT id, username, password, created_at
        FROM users
        ORDER BY id ASC",
    )?;
    let users_iter = stmt.query_map([], |row| {
        Ok(User {
            id: row.get(0)?,
            username: row.get(1)?,
            password: row.get(2)?,
            created_at: row.get(3)?,
        })
    })?;

    let mut users = Vec::new();
    for user in users_iter {
        users.push(user?);
    }

    Ok(users)
}

fn print_usage() {

```

```
        eprintln!(
            "Usage:
cargo run -- seed
cargo run -- register <username> <password>
cargo run -- query <username>
        cargo run -- list
cargo run -- demo"
        );
    }

fn main() -> Result<(), Box<dyn Error>> {
    let db_path = "users.db";
    let conn = Connection::open(db_path)?;
    init_db(&conn)?;

    let mut args = env::args().skip(1);
    match args.next().as_deref() {
        Some("seed") => {
            let inserted = seed_users(&conn)?;
            println!("Seeded {inserted} users into {db_path}.");
        }
        Some("register") => {
            let username = match args.next() {
                Some(value) => value,
                None => {
                    print_usage();
                    return Ok(());
                }
            };
            let password = match args.next() {
                Some(value) => value,
                None => {
                    print_usage();
                    return Ok(());
                }
            };

            if register_user(&conn, &username, &password)? {
                println!("User registered: {username}");
            } else {
                println!("Username already exists: {username}");
            }
        }
    }
}
```

```
    }
  }
  Some("query") => {
    let username = match args.next() {
      Some(value) => value,
      None => {
        print_usage();
        return Ok(());
      }
    };

    match query_user(&conn, &username)? {
      Some(user) => println!(
        "User: id={} username={} password={} created_at\n"
        "={}",
        user.id, user.username, user.password, user.
        created_at
      ),
      None => println!("User not found: {username}"),
    }
  }
  Some("list") => {
    let users = list_users(&conn)?;
    if users.is_empty() {
      println!("No users found.");
    } else {
      for user in users {
        println!(
          "User: id={} username={} password={} created_at\n"
          "={}",
          user.id, user.username, user.password, user.
          created_at
        );
      }
    }
  }
}

Some("demo") | None => {
  seed_users(&conn)?;
  let username = "alice";
  println!("Demo query for username: {username}");
  match query_user(&conn, username)? {
```

```

        Some(user) => println!(
            "User: id={id} username={username} password={password} created_at
            ={}",
            user.id, user.username, user.password, user.
                created_at
        ),
        None => println!("User not found: {username}"),
    }
}

Some(other) => {
    eprintln!("Unknown command: {other}");
    print_usage();
}

Ok(())
}

```

A.2 任务二源代码

```

use mysql::{params, prelude::Queryable, Pool, PooledConn};
use std::env;
use std::error::Error;

fn get_mysql_url() -> String {
    env::var("MYSQL_URL")
        .unwrap_or_else(|_| "mysql://root:root@127.0.0.1:3306/rust_lab
        ".to_string())
}

fn ensure_database(url: &str) -> Result<String, Box<dyn Error>> {
    let opts = mysql::Opts::from_url(url)?;
    let db_name = opts
        .get_db_name()
        .map(|name| name.to_string())
        .unwrap_or_else(|| "rust_lab".to_string());

    let builder = mysql::OptsBuilder::from_opts(opts).db_name(None::<
        String>);
    let pool = Pool::new(builder)?;
}

```

```

    let mut conn = pool.get_conn()?;
    conn.query_drop(format!("CREATE DATABASE IF NOT EXISTS `{}`",
        db_name))?;

    Ok(db_name)
}

fn connect() -> Result<PooledConn, Box<dyn Error>> {
    let url = get_mysql_url();
    let db_name = ensure_database(&url)?;
    let opts = mysql::Opts::from_url(&url)?;
    let builder = mysql::OptsBuilder::from_opts(opts).db_name(Some(
        db_name));
    let pool = Pool::new(builder)?;
    Ok(pool.get_conn())
}

fn init_db(conn: &mut PooledConn) -> mysql::Result<()> {
    conn.query_drop(
        "CREATE TABLE IF NOT EXISTS app_logs (
            log_id INT AUTO_INCREMENT PRIMARY KEY,
            log_level ENUM('INFO','WARN','ERROR') NOT NULL,
            content TEXT NOT NULL,
            created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
        )",
    )?;
    Ok(())
}

fn normalize_level(level: &str) -> Option<String> {
    let upper = level.to_ascii_uppercase();
    match upper.as_str() {
        "INFO" | "WARN" | "ERROR" => Some(upper),
        _ => None,
    }
}

fn insert_log(conn: &mut PooledConn, level: &str, content: &str) ->
mysql::Result<()> {
    conn.exec_drop(
        "INSERT INTO app_logs (log_level, content) VALUES (:level, :

```

```

        content)",
    params! {
        "level" => level,
        "content" => content,
    },
)?;
Ok(())
}

fn seed_logs(conn: &mut PooledConn) -> mysql::Result<()> {
    conn.query_drop(
        "INSERT INTO app_logs (log_level, content, created_at)
        VALUES ('INFO', 'service started', NOW())",
    );
    conn.query_drop(
        "INSERT INTO app_logs (log_level, content, created_at)
        VALUES ('WARN', 'cache miss', NOW() - INTERVAL 2 HOUR)",
    );
    conn.query_drop(
        "INSERT INTO app_logs (log_level, content, created_at)
        VALUES ('ERROR', 'db timeout', NOW() - INTERVAL 1 HOUR)",
    );
    conn.query_drop(
        "INSERT INTO app_logs (log_level, content, created_at)
        VALUES ('INFO', 'health check OK', NOW() - INTERVAL 5 HOUR)",
    );
    conn.query_drop(
        "INSERT INTO app_logs (log_level, content, created_at)
        VALUES ('ERROR', 'legacy error', NOW() - INTERVAL 25 HOUR)",
    );
    Ok(())
}

fn query_recent_errors(conn: &mut PooledConn) -> mysql::Result<Vec<(u64
, String, String, String)>> {
    conn.query_map(
        "SELECT log_id, log_level, content,
            DATE_FORMAT(created_at, '%Y-%m-%d %H:%i:%s') AS
            created_at
        FROM app_logs
        WHERE log_level = 'ERROR'",
    )

```



```

        AND created_at >= NOW() - INTERVAL 24 HOUR
    ORDER BY created_at DESC",
    |(log_id, log_level, content, created_at)| (log_id, log_level,
        content, created_at),
    )
}

fn print_usage() {
    eprintln!(
        "Usage:
cargo run -- init
cargo run -- seed
cargo run -- log <INFO|WARN|ERROR> <content>
cargo run -- recent-errors
cargo run -- demo"
    );
}

fn main() -> Result<(), Box<dyn Error>> {
    let mut conn = connect()?;
    init_db(&mut conn)?;

    let mut args = env::args().skip(1);
    match args.next().as_deref() {
        Some("init") => {
            println!("Table app_logs is ready.");
        }
        Some("seed") => {
            seed_logs(&mut conn)?;
            println!("Seeded sample logs.");
        }
        Some("log") => {
            let level = match args.next() {
                Some(value) => value,
                None => {
                    print_usage();
                    return Ok(());
                }
            };
            let content = match args.next() {
                Some(value) => value,

```

```

        None => {
            print_usage();
            return Ok(());
        }
    };

    let level = match normalize_level(&level) {
        Some(value) => value,
        None => {
            eprintln!("Invalid level: {level}");
            return Ok(());
        }
    };

    insert_log(&mut conn, &level, &content)?;
    println!("Inserted log: level={level} content={content}");
}

Some("recent-errors") => {
    let rows = query_recent_errors(&mut conn)?;
    if rows.is_empty() {
        println!("No ERROR logs in the last 24 hours.");
    } else {
        for (log_id, log_level, content, created_at) in rows {
            println!(
                "log_id={} level={} created_at={} content={}",
                log_id, log_level, created_at, content
            );
        }
    }
}

Some("demo") => {
    seed_logs(&mut conn)?;
    insert_log(&mut conn, "INFO", "demo info"?);
    insert_log(&mut conn, "WARN", "demo warn"?);
    insert_log(&mut conn, "ERROR", "demo error"?);

    let rows = query_recent_errors(&mut conn)?;
    println!("Recent ERROR logs (last 24 hours):");
    if rows.is_empty() {
        println!("No ERROR logs in the last 24 hours.");
    } else {

```

```

        for (log_id, log_level, content, created_at) in rows {
            println!(
                "log_id={} level={} created_at={} content={}",
                log_id, log_level, created_at, content
            );
        }
    }
}

Some(other) => {
    eprintln!("Unknown command: {other}");
    print_usage();
}

None => {
    print_usage();
}
}

Ok(())
}

```

A.3 任务三源代码

```

use postgres::{Client, NoTls};
use std::env;
use std::error::Error;

fn get_postgres_url() -> String {
    env::var("POSTGRES_URL")
        .unwrap_or_else(|_| "postgres://postgres:postgres@127
            .0.0.1:5432/rust_lab".to_string())
}

fn split_postgres_url(url: &str) -> Result<(String, String, String),
    Box<dyn Error>> {
    let (main, params) = match url.split_once('?') {
        Some((left, right)) => (left, right),
        None => (url, ""),
    };
    let slash_pos = main
        .rfind('/')

```

```

        .ok_or("POSTGRES_URL must include a database name")?;
let base = &main[..slash_pos];
let db_part = &main[slash_pos + 1..];
let db_name = if db_part.is_empty() { "rust_lab" } else { db_part
    };
let params_suffix = if params.is_empty() {
    String::new()
} else {
    format!("{}", params)
};

Ok((base.to_string(), db_name.to_string(), params_suffix))
}

fn build_postgres_url(base: &str, db_name: &str, params: &str) ->
String {
    format!("{}/{}/{}", base, db_name, params)
}

fn escape_identifier(value: &str) -> String {
    value.replace("'", "\"")
}

fn ensure_database(url: &str) -> Result<String, Box<dyn Error>> {
    let (base, db_name, params) = split_postgres_url(url)?;
    let admin_url = build_postgres_url(&base, "postgres", &params);
    let mut admin = Client::connect(&admin_url, NoTls)?;

    let rows = admin.query(
        "SELECT 1 FROM pg_database WHERE datname = $1",
        [&db_name],
    )?;
    if rows.is_empty() {
        let escaped = escape_identifier(&db_name);
        admin.batch_execute(&format!("CREATE DATABASE \"{}\"", escaped))
            ?;
    }

    Ok(db_name)
}

```

```
fn connect() -> Result<Client, Box<dyn Error>> {
    let url = get_postgres_url();
    let db_name = ensure_database(&url)?;
    let (base, _db_name, params) = split_postgres_url(&url)?;
    let db_url = build_postgres_url(&base, &db_name, &params);
    Ok(Client::connect(&db_url, NoTls)?)
}

fn init_db(client: &mut Client) -> Result<(), postgres::Error> {
    client.batch_execute(
        "CREATE TABLE IF NOT EXISTS products (
            product_id SERIAL PRIMARY KEY,
            name TEXT NOT NULL,
            stock INT NOT NULL,
            updated_at TIMESTAMP NOT NULL DEFAULT NOW()
        )",
    )?;
    Ok(())
}

fn seed_products(client: &mut Client) -> Result<(), postgres::Error> {
    let samples = [
        ("Keyboard", 25),
        ("Mouse", 8),
        ("Monitor", 12),
        ("USB Cable", 5),
        ("Laptop Stand", 9),
    ];

    for (name, stock) in samples {
        client.execute(
            "INSERT INTO products (name, stock) VALUES ($1, $2)",
            [&name, &stock],
        )?;
    }

    Ok(())
}

fn purchase_product(client: &mut Client, product_id: i32, qty: i32) ->
    Result<(), postgres::Error> {
```

```

let row = client.query_opt(
    "UPDATE products
    SET stock = stock - $1,
        updated_at = NOW()
    WHERE product_id = $2
    AND stock >= $1
    RETURNING product_id, name, stock, to_char(updated_at, 'YYYY-MM-DD HH24:MI:SS')",
    &[&qty, &product_id],
)?;

match row {
    Some(row) => {
        let id: i32 = row.get(0);
        let name: String = row.get(1);
        let stock: i32 = row.get(2);
        let updated_at: String = row.get(3);
        println!(
            "Purchase OK: id={} name={} stock={} updated_at=",
            id, name, stock, updated_at
        );
    }
    None => {
        println!(
            "Purchase failed: product_id={} qty={} (insufficient
            stock or not found)",
            product_id, qty
        );
    }
}

Ok(())
}

fn query_low_stock(
    client: &mut Client,
    threshold: i32,
) -> Result<Vec<(i32, String, i32, String)>, postgres::Error> {
    let rows = client.query(
        "SELECT product_id, name, stock, to_char(updated_at, 'YYYY-MM-DD HH24:MI:SS')

```

```

        FROM products
        WHERE stock < $1
        ORDER BY stock ASC",
        &[&threshold],
    )?;

Ok(rows
    .into_iter()
    .map(|row| {
        let id: i32 = row.get(0);
        let name: String = row.get(1);
        let stock: i32 = row.get(2);
        let updated_at: String = row.get(3);
        (id, name, stock, updated_at)
    })
    .collect())
}

fn print_usage() {
    eprintln!(
        "Usage:
cargo run -- init
cargo run -- seed
cargo run -- purchase <product_id> <qty>
cargo run -- low-stock <threshold>
cargo run -- demo"
    );
}

fn main() -> Result<(), Box<dyn Error>> {
    let mut client = connect()?;
    init_db(&mut client)?;

    let mut args = env::args().skip(1);
    match args.next().as_deref() {
        Some("init") => {
            println!("Table products is ready.");
        }
        Some("seed") => {
            seed_products(&mut client)?;
            println!("Seeded sample products.");
        }
        _ => {}
    }
}

```

```

}
Some("purchase") => {
    let product_id: i32 = match args.next() {
        Some(value) => value.parse().unwrap_or(0),
        None => {
            print_usage();
            return Ok(());
        }
    };
    let qty: i32 = match args.next() {
        Some(value) => value.parse().unwrap_or(0),
        None => {
            print_usage();
            return Ok(());
        }
    };

    if product_id <= 0 || qty <= 0 {
        eprintln!("product_id and qty must be positive.");
        return Ok(());
    }

    purchase_product(&mut client, product_id, qty)?;
}
Some("low-stock") => {
    let threshold: i32 = match args.next() {
        Some(value) => value.parse().unwrap_or(10),
        None => {
            print_usage();
            return Ok(());
        }
    };

    let rows = query_low_stock(&mut client, threshold)?;
    if rows.is_empty() {
        println!("No products with stock below {threshold}.");
    } else {
        for (id, name, stock, updated_at) in rows {
            println!(
                "product_id={} name={} stock={} updated_at={}",
                id, name, stock, updated_at
            );
        }
    }
}

```



```

        );
    }
}

Some("demo") => {
    seed_products(&mut client)?;
    purchase_product(&mut client, 1, 20)?;

    let rows = query_low_stock(&mut client, 10)?;
    if rows.is_empty() {
        println!("No products with stock below 10.");
    } else {
        for (id, name, stock, updated_at) in rows {
            println!(
                "product_id={} name={} stock={} updated_at={}",
                id, name, stock, updated_at
            );
        }
    }
}

Some(other) => {
    eprintln!("Unknown command: {other}");
    print_usage();
}

None => {
    print_usage();
}

}

Ok(())
}

```

A.4 任务四源代码

```

use mysql::{params, prelude::Queryable, Pool, PooledConn};
use std::env;
use std::error::Error;

fn get_mysql_url() -> String {
    env::var("MYSQL_URL")
}

```

```

        .unwrap_or_else(|_| "mysql://root:root@127.0.0.1:3306/rust_lab"
            .to_string())
    }

fn ensure_database(url: &str) -> Result<String, Box<dyn Error>> {
    let opts = mysql::Opts::from_url(url)?;
    let db_name = opts
        .get_db_name()
        .map(|name| name.to_string())
        .unwrap_or_else(|| "rust_lab".to_string());

    let builder = mysql::OptsBuilder::from_opts(opts).db_name(None:<
        String>);
    let pool = Pool::new(builder)?;
    let mut conn = pool.get_conn()?;
    conn.query_drop(format!("CREATE DATABASE IF NOT EXISTS `{}`",
        db_name))?;

    Ok(db_name)
}

fn connect() -> Result<PooledConn, Box<dyn Error>> {
    let url = get_mysql_url();
    let db_name = ensure_database(&url)?;
    let opts = mysql::Opts::from_url(&url)?;
    let builder = mysql::OptsBuilder::from_opts(opts).db_name(Some(
        db_name));
    let pool = Pool::new(builder)?;
    Ok(pool.get_conn())
}

fn init_db(conn: &mut PooledConn) -> mysql::Result<()> {
    conn.query_drop(
        "CREATE TABLE IF NOT EXISTS orders (
            order_id INT AUTO_INCREMENT PRIMARY KEY,
            amount DECIMAL(10, 2) NOT NULL,
            created_at DATETIME NOT NULL
        )",
    )?;

    conn.query_drop(

```

```

        "CREATE OR REPLACE VIEW daily_sales_report AS
        SELECT DATE(created_at) AS report_date,
               COUNT(*) AS order_count,
               SUM(amount) AS total_amount,
               AVG(amount) AS avg_amount
        FROM orders
        GROUP BY DATE(created_at)",
    )?;

    Ok(())
}

fn seed_orders(conn: &mut PooledConn) -> mysql::Result<()> {
    conn.query_drop(
        "INSERT INTO orders (amount, created_at)
        VALUES (120.50, NOW())",
    )?;
    conn.query_drop(
        "INSERT INTO orders (amount, created_at)
        VALUES (75.00, NOW() - INTERVAL 1 DAY)",
    )?;
    conn.query_drop(
        "INSERT INTO orders (amount, created_at)
        VALUES (260.00, NOW() - INTERVAL 1 DAY)",
    )?;
    conn.query_drop(
        "INSERT INTO orders (amount, created_at)
        VALUES (30.99, NOW() - INTERVAL 2 DAY)",
    )?;
    conn.query_drop(
        "INSERT INTO orders (amount, created_at)
        VALUES (410.20, NOW() - INTERVAL 3 DAY)",
    )?;
    Ok(())
}

fn query_report(
    conn: &mut PooledConn,
    start_date: &str,
    end_date: &str,
) -> mysql::Result<Vec<(String, u64, String, String)>> {

```

```

    conn.exec_map(
        "SELECT DATE_FORMAT(report_date, '%Y-%m-%d') AS report_date,
            order_count,
            CAST(total_amount AS CHAR) AS total_amount,
            CAST(avg_amount AS CHAR) AS avg_amount
        FROM daily_sales_report
        WHERE report_date BETWEEN :start_date AND :end_date
        ORDER BY report_date",
        params! {
            "start_date" => start_date,
            "end_date" => end_date,
        },
        |(report_date, order_count, total_amount, avg_amount)| {
            (report_date, order_count, total_amount, avg_amount)
        },
    )
}

fn print_usage() {
    eprintln!(
        "Usage:
cargo run -- init
cargo run -- seed
cargo run -- report <YYYY-MM-DD> <YYYY-MM-DD>
cargo run -- demo"
    );
}

fn main() -> Result<(), Box<dyn Error>> {
    let mut conn = connect()?;
    init_db(&mut conn)?;

    let mut args = env::args().skip(1);
    match args.next().as_deref() {
        Some("init") => {
            println!("Table orders and view daily_sales_report are
                ready.");
        }
        Some("seed") => {
            seed_orders(&mut conn)?;
            println!("Seeded sample orders.");
        }
        _ => {}
    }
}

```

```
}
Some("report") => {
    let start_date = match args.next() {
        Some(value) => value,
        None => {
            print_usage();
            return Ok(());
        }
    };
    let end_date = match args.next() {
        Some(value) => value,
        None => {
            print_usage();
            return Ok(());
        }
    };

    let rows = query_report(&mut conn, &start_date, &end_date)
        ?;
    if rows.is_empty() {
        println!("No report rows for {start_date} to {end_date}
            .");
    } else {
        for (report_date, order_count, total_amount, avg_amount
            ) in rows {
            println!(
                "date={} orders={} total={} avg={}",
                report_date, order_count, total_amount,
                avg_amount
            );
        }
    }
}

Some("demo") => {
    seed_orders(&mut conn)?;
    let start_date = "2000-01-01";
    let end_date = "2100-01-01";

    let rows = query_report(&mut conn, start_date, end_date)?;
    if rows.is_empty() {
        println!("No report rows for {start_date} to {end_date}
```

```
        }.");
    } else {
        for (report_date, order_count, total_amount, avg_amount
            ) in rows {
            println!(
                "date={} orders={} total={} avg={}",
                report_date, order_count, total_amount,
                avg_amount
            );
        }
    }
}

Some(other) => {
    eprintln!("Unknown command: {other}");
    print_usage();
}

None => {
    print_usage();
}

}

Ok(())
}
```

A.5 运行命令

启动数据库 (Docker)

```
cd Rust/Lab-3
```

```
docker-compose up -d
```

任务一

```
cd task1_user_registration_sqlite
```

```
cargo run -- demo
```

任务二

```
cd ../task2_log_mysql
```

```
cargo run -- demo
```

任务三

```
cd ../task3_inventory_postgres  
cargo run -- demo
```

任务四

```
cd ../task4_report_mysql  
cargo run -- demo
```